7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

1 Introduzione **5**
1.1 Obiettivo del documento 5
1.2 Glossario 5
1.3 Riferimenti 5
1.3.1 Riferimenti normativi 5
1.3.2 Riferimenti informativi 5

2 Tecnologie utilizzate **7**

3 Linguaggio di Programmazione: TypeScript **7**
3.1 Deployment: Electron 7
3.2 Framework Frontend: Angular 8
3.3 Librerie Aggiuntive 9
3.4 Tecnologie di analisi statica 10
3.5 Tecnologie di analisi dinamica 10

4 Contesto del sistema **10**
4.1 Modello architetturale 11
4.2 Descrizione degli attori esterni 11

5 Architettura dei container **12**
5.1 Renderer 13
5.1.1 Struttura architetturale 13
5.2 [Main] 13
5.2.1 Tecnologia: Node.js 13

6 Architettura delle componenti **14**
6.1 [Componente 1] 14
6.1.1 Descrizione 14
6.1.2 Interfacce 14
6.1.3 Dipendenze 14
6.1.4 Tracciamento dei requisiti 14
6.2 [Componente 2] 14
6.2.1 Descrizione 14
6.2.2 Interfacce 14
6.2.3 Dipendenze 14
6.2.4 Tracciamento dei requisiti 14
6.3 Componenti di Ricerca (FrontEnd) 15
6.3.1 Architettura generale 15
6.3.2 Layer di gestione dello stato e validazione 18
6.3.3 Layer di visualizzazione dei risultati 22
6.3.4 Layer di ricerca globale 25
6.3.5 Conformità normativa e linee guida 26

6.3.6 Comunicazione IPC e Gateway Layer 27
6.3.7 DTO e Trasformazione Dati 29
6.4 Validazione Filtri 30

List of Figures

1 Diagramma C1 - Contesto del sistema *DIPReader* 11
2 Diagramma C2 - Container del sistema *DIPReader* 12
3 Diagramma C3 - Data flow della funzionalità di ricerca 16
4 Diagramma C4 - AdvancedFilterPanel e gestione dinamica dei filtri . . . 18
5 Diagramma C4 - SUbjectsFilterComponent e SubjectType Strategy Registry 20
6 Diagramma C4 - SearchResultFactory e componenti di visualizzazione specializzati 23
7 Diagramma C4 - SearchBarComponent e SearchBarRulesManager 25
8 Diagramma C4 - SearchIpcGateway e comunicazione IPC 27
9 Diagramma C4 - FilterValidatorService e FilterValidatorFn 30

List of Tables

2 Dettagli sulla scelta di TypeScript come linguaggio di programmazione . 7
4 Dettagli sulla scelta di Electron come piattaforma di deployment 7
6 Dettagli sulla scelta di Angular come framework frontend 8
8 Dettagli sulle scelte di librerie aggiuntive 9
10 Dettagli sulle scelte di tecnologie di analisi statica 10
12 Dettagli sulle scelte di tecnologie di analisi dinamica 10

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

La scelta delle teccnologie è stata guidata dalla necessità di rispetta i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

3 Linguaggio di Programmazione: TypeScript

Scelta	Versione	Motivazione
TypeScript	5.9.2	Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron.

Table 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

3.1 Deployment: Electron

Scelta	Versione	Motivazione
Electron	30.0.0	Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti.

Table 4: Dettagli sulla scelta di Electron come piattaforma di deployment

3.2 Framework Frontend: Angular

Scelta	Versione	Motivazione
Angular	21.2.5	Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice.

Table 6: Dettagli sulla scelta di Angular come framework frontend

3.3 Librerie Aggiuntive

Scelta	Versione	Motivazione
Better SQLite	12.8.0	SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto.
sqlite-vss	0.1.2	Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema.
ONNX Runtime	1.17.3	Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto.
Transformers.js	3.8.1	Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto.

Table 8: Dettagli sulle scelte di librerie aggiuntive

3.4 Tecnologie di analisi statica

Scelta	Versione	Motivazione
SonarQube	2026.2	Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile.
Prettier	2.9.3	Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme.

Table 10: Dettagli sulle scelte di tecnologie di analisi statica

3.5 Tecnologie di analisi dinamica

Scelta	Versione	Motivazione
Vitest	2.3.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock.
Playwright	1.59.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E.

Table 12: Dettagli sulle scelte di tecnologie di analisi dinamica

4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP. STAMPA DA VEDERE SE HA SENSO METTERLO

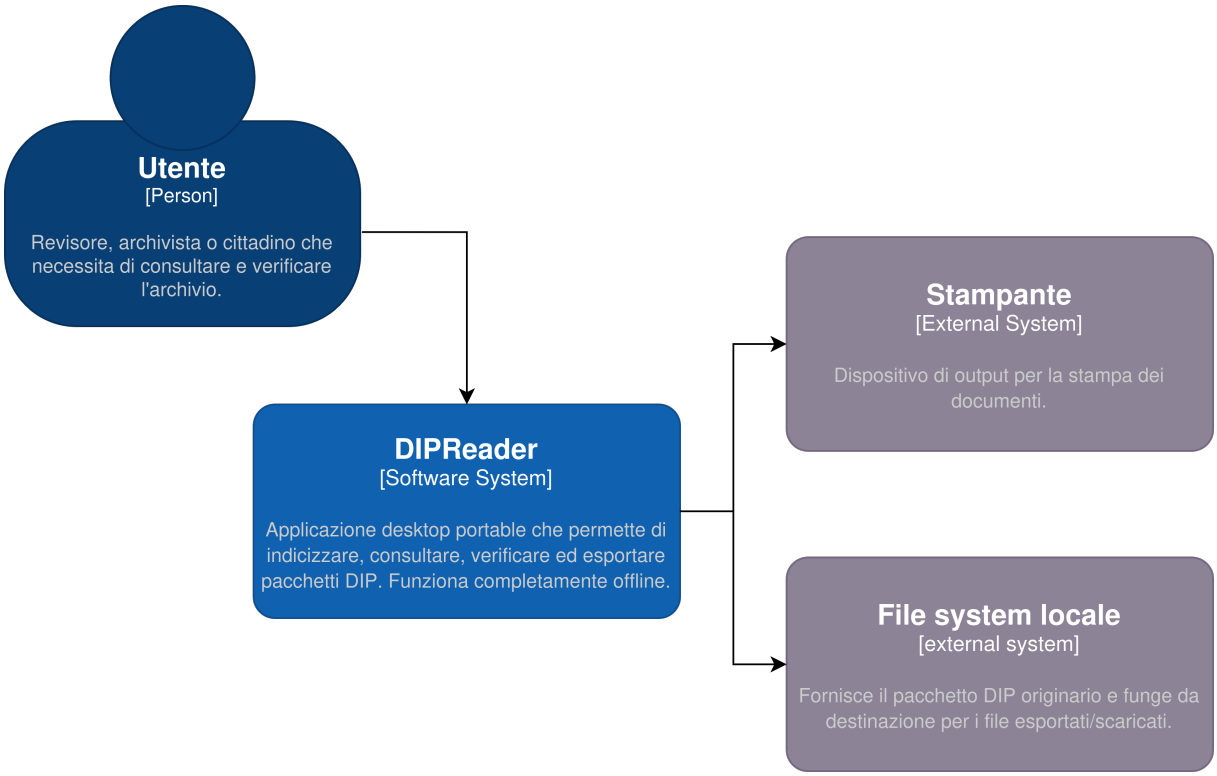

Figure 1: Diagramma C1 - Contesto del sistema *DIPReader*

4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- STAMPA DA VEDERE SE HA SENSO METTERLO

5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema così facendo permette di avere un backend facilmente estendibile, modificabile e soprattutto che si adatta allla struttura del DIP autonomamente. D'altra parte il frontend è completamente disaccoppiato ma strutturato in modo che i componenti di presentazione siano facilmente sostituibili, modificabili e soprattutto adattabili a nuove esigenze.

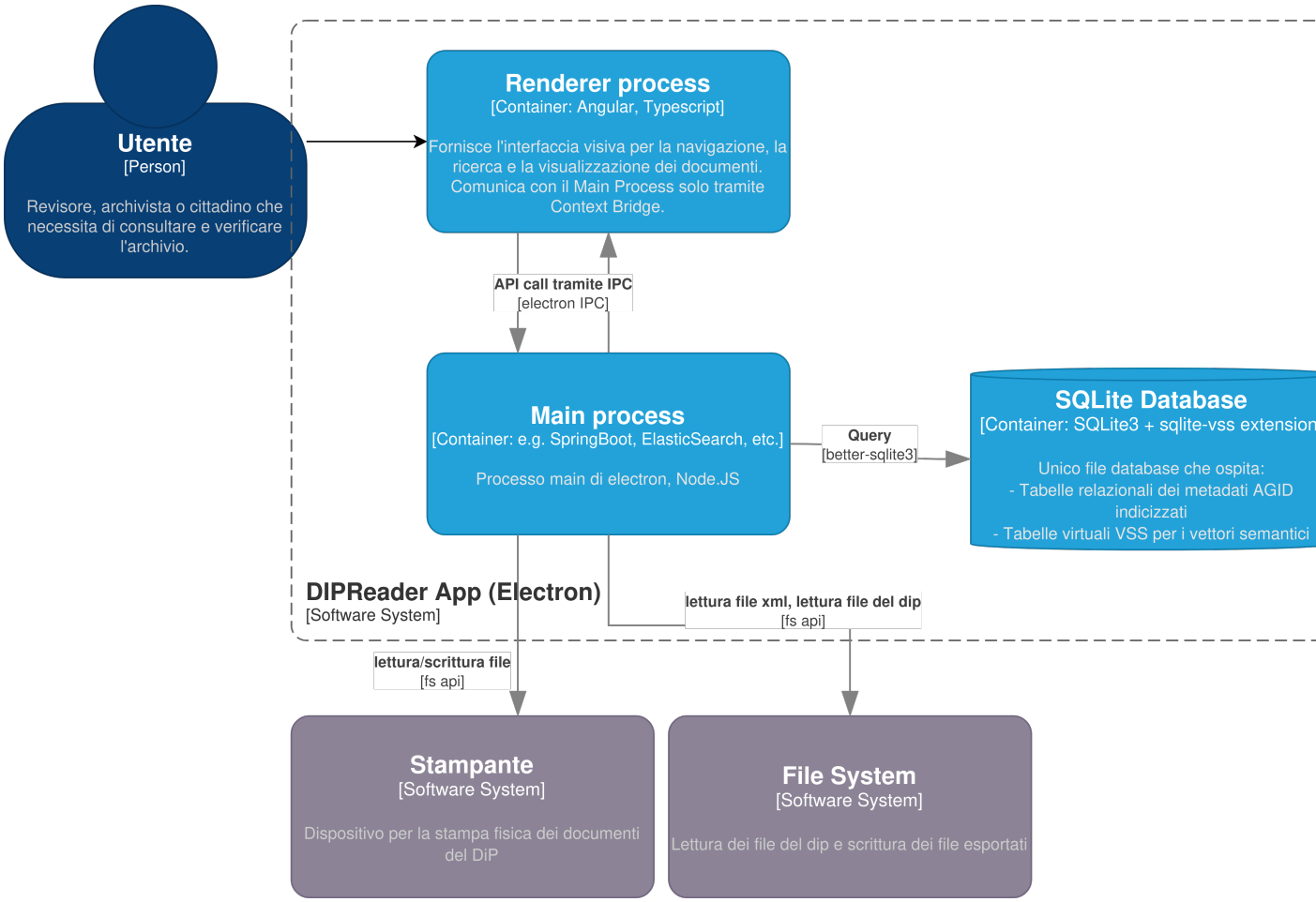

Figure 2: Diagramma C2 - Container del sistema *DIPReader*

5.1 Renderer

5.1.1 Struttura architetturale

Il container *Renderer* è responsabile della presentazione dell'interfaccia utente e dell'interazione con l'utente. Per la gestione quindi dell'interfaccia è stato scelto di strutturare il container per componenti. Ogni componente a sua volta è strutturato da una serie di services che si occupano di gestire la logica di presentazione, e da una serie di componenti di presentazione che si occupano di mostrare i dati all'utente.
Visto poi che le interazioni tra i diversi componenti di presentazione sono limitate, è stato scelto di strutturare il container in modo che ogni componente sia completamente disaccoppiato dagli altri. Gli unici punti di contatto risultano infatti essere il sistema di routing e i canali IPC, che permettono ai componenti di comunicare tra loro e con il backend.
In presenza poi di componenti come per esempio l'esportazione dei file, che richiedono di essere inseriti in più punti del sistema, è stato scelto di strutturare il container in modo che questi componenti siano facilmente integrabili in qualsiasi punto del sistema, senza dover modificare la struttura del container o dei componenti di presentazione grazie a delle semplici Injection di dipendenze.
È poi stato applicato il pattern Smart and Dumb Components, che permette di separare la logica di presentazione pura dalla logica di business che la coordina.

5.2 [Main]

5.2.1 Tecnologia: Node.js

Descrizione

6 Architettura delle componenti

6.1 [Componente 1]

6.1.1 Descrizione

6.1.2 Interfacce

6.1.3 Dipendenze

6.1.4 Tracciamento dei requisiti

6.2 [Componente 2]

6.2.1 Descrizione

6.2.2 Interfacce

6.2.3 Dipendenze

6.2.4 Tracciamento dei requisiti

6.3 Componenti di Ricerca (FrontEnd)

6.3.1 Architettura generale

La sezione di ricerca implementa un'architettura stratificata basata su due pattern fondamentali:

- **Smart & Dumb Components**: separazione netta tra logica (SearchPageComponent) e presentazione (componenti figli).
- **Facade Pattern**: la SearchFacade astrae la complessità dell'orchestrazione dei servizi, fornendo un'interfaccia semplificata al layer di presentazione.

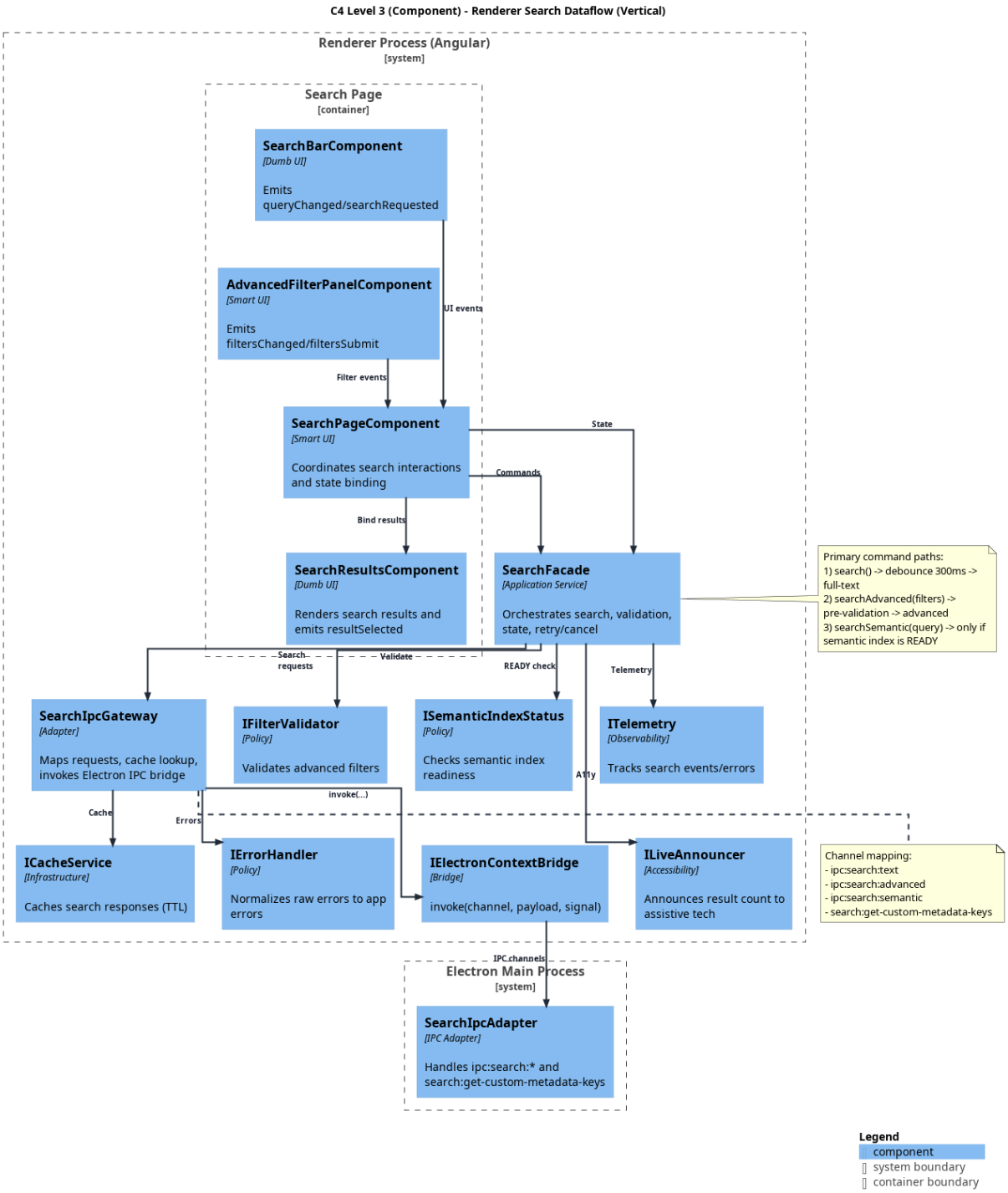

Figure 3: Diagramma C3 - Data flow della funzionalità di ricerca

La SearchPageComponent è il componente radice che:

1. Risolve le dipendenze tramite Angular Dependency Injection (iniettando *interfacce* di servizi, non implementazioni concrete, applicando il principio di sostituzione di Liskov).

2. Delega alla SearchFacade tutta la logica di orchestrazione tra servizi: validazione, gestione dello stato, routing.

3. Espone al template esclusivamente `@Input`/`@Output` properties, mantenendo la componente come contenitore logico puro.

Questo approccio garantisce:

- **Testabilità**: ogni componente e servizio è mockabile in isolamento.
- **Manutenibilità**: i cambiamenti ai servizi non propagano ai componenti di presentazione.
- **Riutilizzabilità**: i componenti dumb possono essere utilizzati in contesti diversi.

6.3.2 Layer di gestione dello stato e validazione

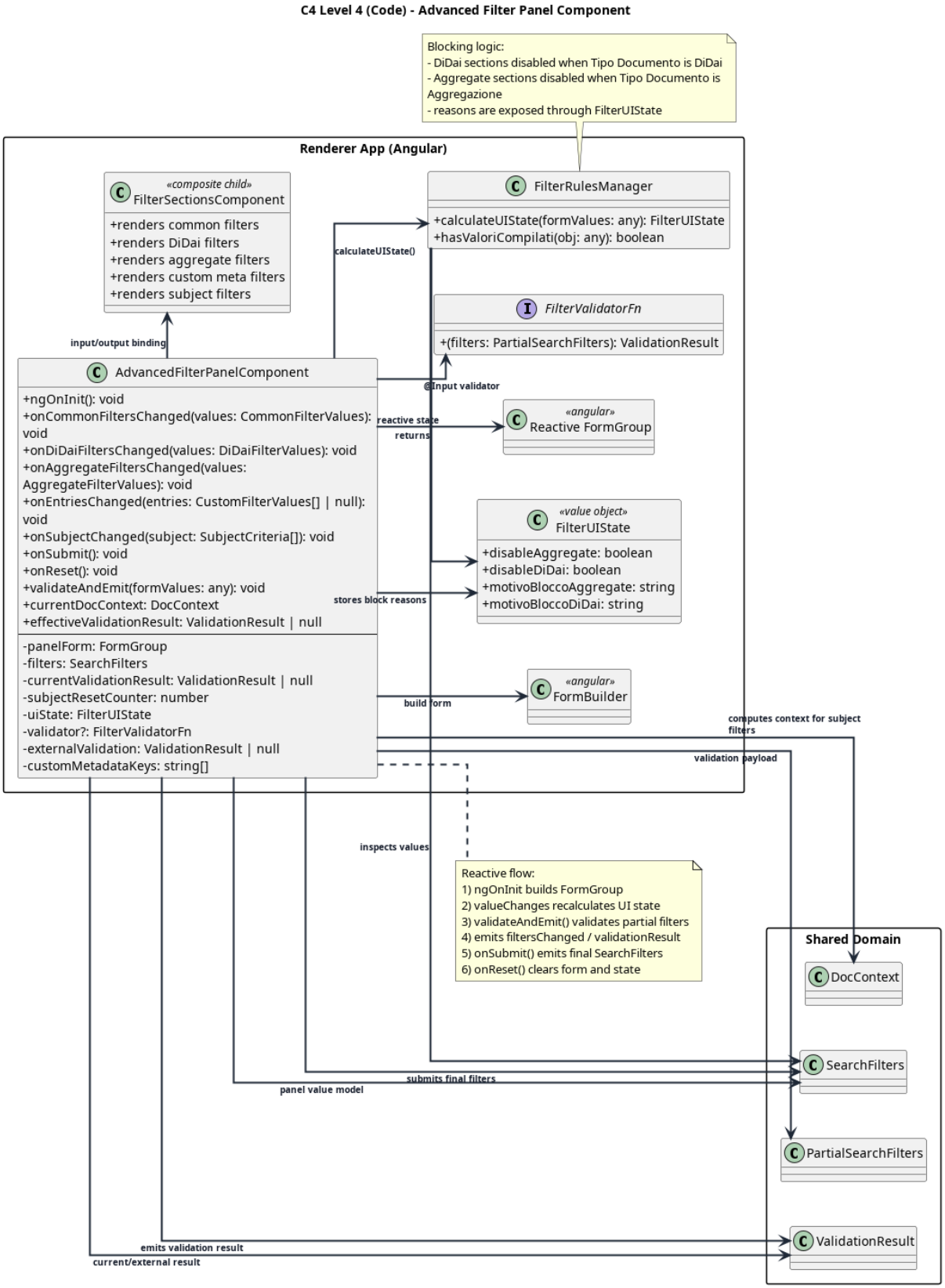

Figure 4: Diagramma C4 - AdvancedFilterPanel e gestione dinamica dei filtri

AdvancedFilterPanelComponent e FilterRulesManager Il pannello filtri avanzati è responsabile della raccolta e gestione dello stato complessivo della ricerca. La sua responsabilità core è:

- Mantenere lo stato mutabile dei filtri durante l'interazione utente.
- Coordinare i sotto-componenti (CommonFiltersComponent, DiDaiFiltersComponent, AggregateFiltersComponent, SubjectsFilterComponent, CustomMetadataComponent).
- Applicare regole di validazione dinamiche tramite il FilterRulesManager.

Il *FilterRulesManager* implementa la logica di validazione contestuale: poiché i campi pertinenti variano in base al tipo di documento selezionato (identificato dal DocContext), questo componente:

- Definisce quali filtri sono visibili, obbligatori o opzionali.
- Enforces vincoli di coerenza (ad es., un campo di un tipo specifico non deve essere compilato se quel tipo non è selezionato).
- Può disabilitare/abilitare dynamicamente i campi senza ricaricare il DOM, garantendo UX fluido.

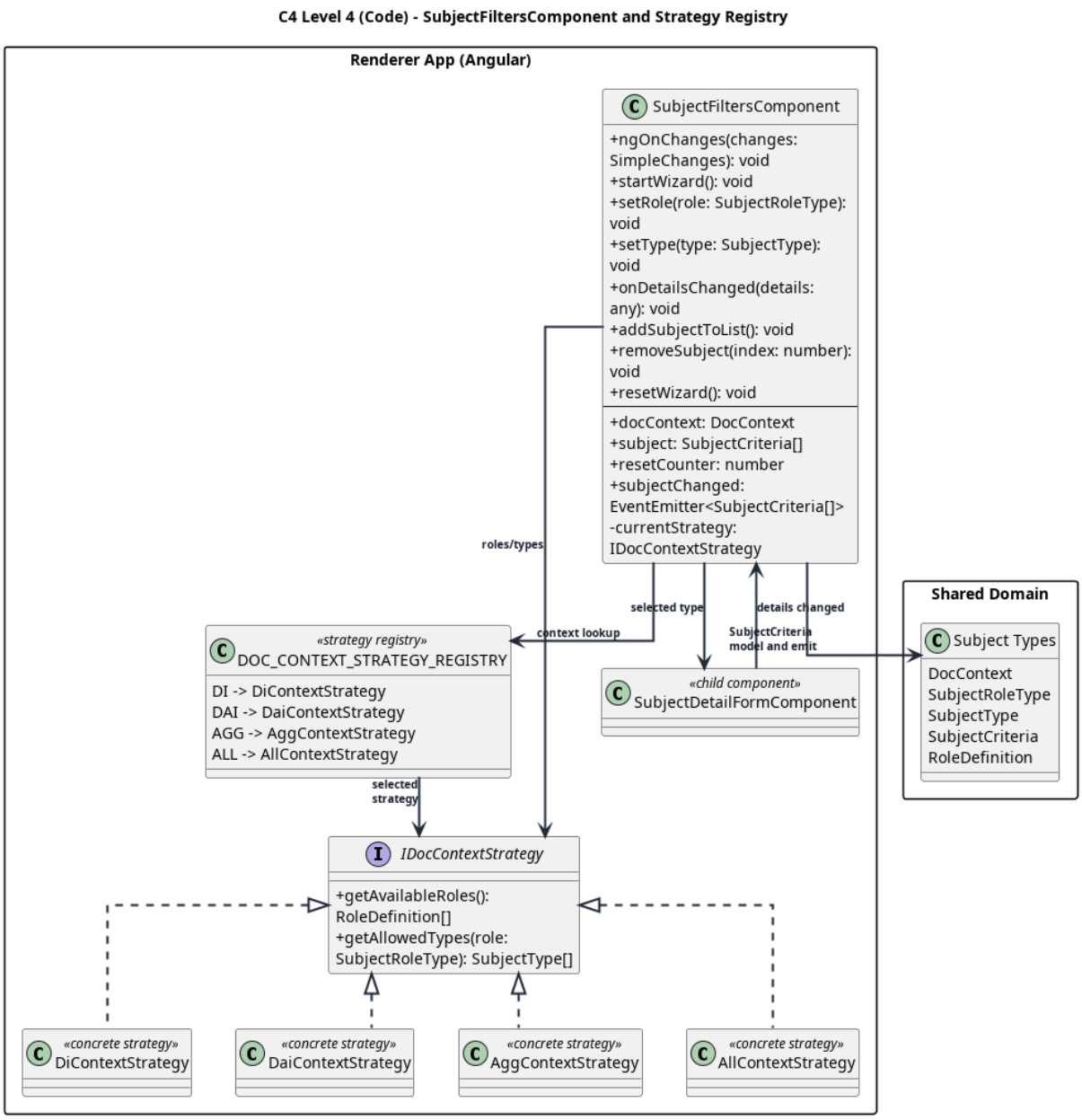

Figure 5: Diagramma C4 - SUbjectsFilterComponent e SubjectType Strategy Registry

SubjectsFilterComponent e Strategy Registry Il filtro dei soggetti è strutturato secondo il pattern Strategy per consentire comportamenti specializzati per ogni contesto documentale:

1. **Role Strategy Registry**: in base al DocContext e al tipo di documento, viene selezionata la configurazione di ruoli consentiti. Ad esempio, un documento DiPIndex può avere ruoli (creatore, responsabile) diversi da un'aggregazione (responsabile della procedura, responsabile di fase).

2. **SubjectType Detail Field Registry**: per ogni SubjectType (persona fisica, persona giuridica, ente pubblico), viene mantenuta una registry che associa il tipo ai campi di dettaglio richiesti. Questa consente la generazione *dichiarativa* del form, evitando logiche condizionali sparse nel codice.

3. **Wizard guidato**: il componente orchestr a un flusso step-by-step:
    - Selezione del ruolo (reso disponibile dal Role Strategy).
    - Selezione del SubjectType.
    - Compilazione dinamica dei campi di dettaglio.
    - Raccolta come *SubjectCriteria* e integrazione nello stato globale dei filtri.

In fase di esecuzione della ricerca, i SubjectCriteria vengono trasformati in una struttura gerarchica di query compatibile col back-end, mantenendo una separazione netta tra:

- **Modello di interfaccia** (SubjectCriteria): forma usabile in UI.
- **Modello di ricerca** (QueryModel per back-end): forma serializzabile e ottimizzata per la query.

Ciò consente:

- Facile estensione: nuovi SubjectType o ruoli si aggiungono alle registry senza modificare il componente.
- Manutenzione centralizzata: le regole di validazione e composizione dei filtri risiedono in un'unica registry, non sparse nel codice.
- Coerenza funzionale: un cambiamento nel back-end (nuovo ruolo, nuovo tipo di soggetto) si riflette solo nelle registry, non nella logica del componente.

Componenti di filtro semplici La maggior parte dei filtri sono componenti dumb di presentazione, allineati alle specifiche AGID e alle linee guida Sanmarco Informatica:

CommonFiltersComponent Gestisce i filtri applicabili a **tutti** i tipi di documento:

- Anni di Conservazione (anno inizio, anno fine).
- Chiave descrittiva (campo di testo libero).
- Classificazione con sottocampi (partizione, categoria, codice).

Lo schema è stratificato: la validazione della classificazione (es., "codice" disponibile solo se "partizione" è compilata) è delegata al FilterRulesManager, non al componente.

DiDaiFiltersComponent Gestisce i filtri specifici per Documenti Informatici (DiI) e Documenti Amministrativi Informatici (DAI):

- Tipologia di documento (enum ristretto ai soli tipi DiI/DAI).
- Modalità di formazione (pre-formati, post-formati, cartacei scansionati, ...).
- Nome, versione, ID del documento.
- Dati di formato (tipologia MIME, versione, estensione).

- Verifica e tracciatura: hash algoritmo, hash valore, data tracciatura, modifiche rilevate.

Ciascun campo è memoizzato e reattivo: il componente emette update solo quando il valore effettivamente cambia.

AggregateFiltersComponent Gestisce i filtri specifici per Aggregazioni Documentali (DiP):

- Tipologia aggregazione (fascicolo, procedimento, cartella).
- ID aggregazione.
- Date di apertura e chiusura.
- Dati di procedimento: numero, data inizio, fase corrente.
- Dati di assegnazione: assegnatario, data assegnazione.

CustomMetadataComponent I metadati custom sono coppie chiave-valore personalizzate non previste dallo schema AGID. La loro gestione presenta sfide specifiche:

1. **Discovery dinamico**: le chiavi disponibili devono riflettere l'insieme di chiavi *effettivamente presenti* nei documenti del repository, non uno schema prefissato.
2. **Implementazione**: il componente:
   - Interroga un servizio (GetCustomMetadataKeysUC dal back-end) per recuperare la lista di chiavi popolate.
   - Popola un'autocomplete o una select renderizzata dinamicamente.
   - Consente all'utente di filtrare documenti per chiave e valore.
   - Valida l'input (es., no valori null, lunghezza massima).
3. **Separazione di responsabilità**: il componente UI gestisce *solo* la presentazione; il servizio di ricerca si occupa di trasformare i criteri custom nel modello di query del back-end.

6.3.3 Layer di visualizzazione dei risultati

SearchResultFactory e Dynamic Component Instantiation La visualizzazione dei risultati di ricerca deve adattarsi al tipo di entità ritornata. Poiché processi, classi, aggregazioni e documenti hanno strutture dati e esigenze di presentazione radicalmente diverse, è stato adottato il pattern *Factory* con registry dinamico:

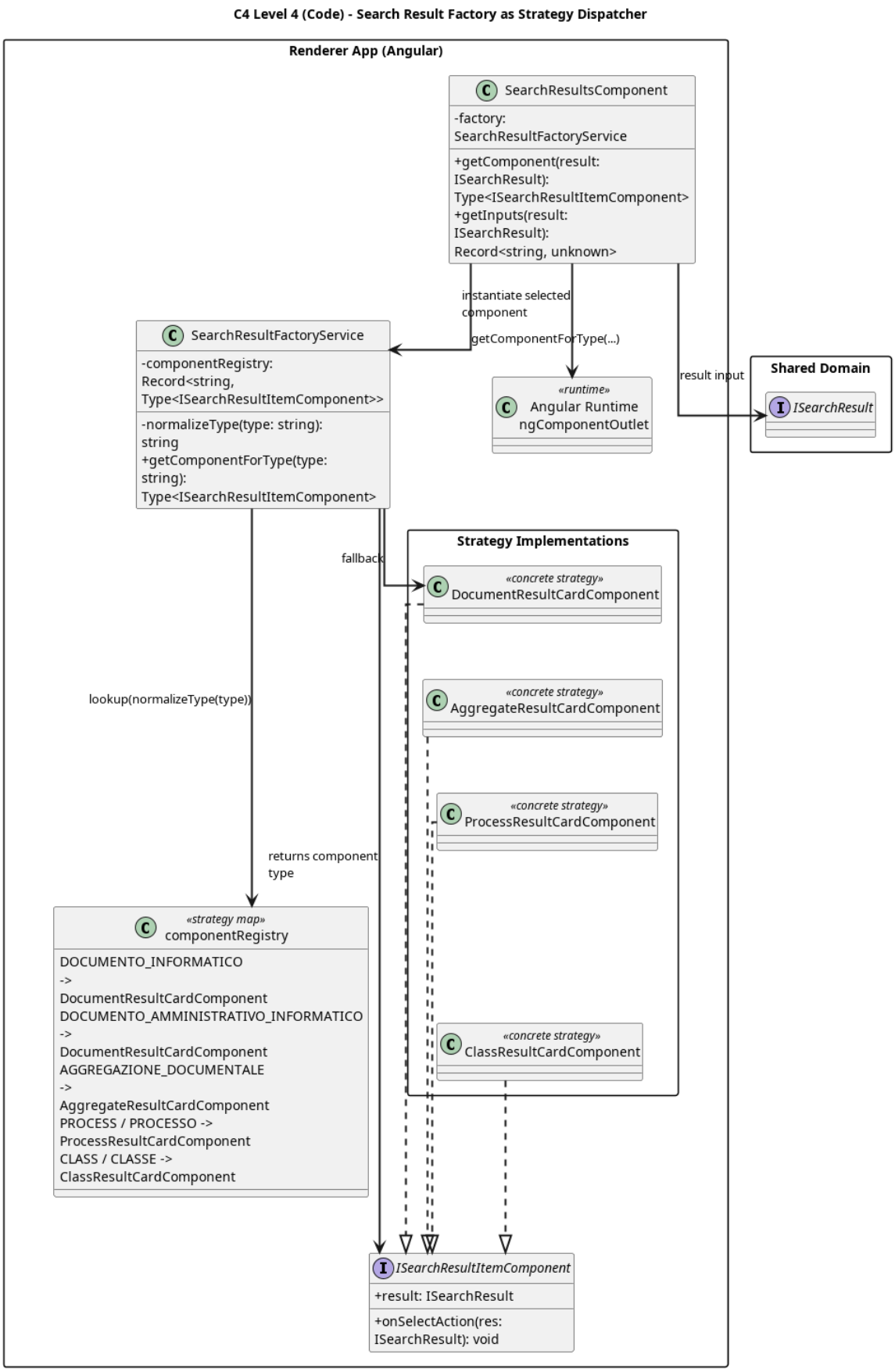

Figure 6: Diagramma C4 - SearchResultFactory e componenti di visualizzazione specializzati

1. **SearchResultFactory**: componente factory che:
   - Interroga il tipo di risultato ricevuto (DocumentResult, ProcessResult, AggregateResult, ClassResult).
   - Consulta una registry configurata *a compile-time* che associa tipo → componente di visualizzazione.
   - Istanzia e proietta il componente corretto nel template tramite *ng-container* e *ngComponentOutlet*.
2. **Componenti di visualizzazione specializzati**:

   ProcessResultComponent Visualizza informazioni essenziali del processo:
   - ID processo (copiabile).
   - Descrizione breve.
   - Numero documenti associati.
   - Link all'apertura del processo.

   No azioni di download/stampa (non applicabili ai processi).

   DocumentResultComponent Visualizza metadati e azioni specifiche per documenti:
   - Anteprima miniatura (se disponibile).
   - Titolo, descrizione, formato.
   - Data creazione, autore (da metadati AGID).
   - Azioni contestuali: apri anteprima, scarica, stampa, mostra metadati.

   AggregateResultComponent Visualizza dati di aggregazione:
   - Tipo aggregazione (fascicolo/procedimento).
   - ID e data apertura.
   - Numero documenti e sotto-aggregazioni.
   - Link all'apertura dell'aggregazione.

   ClassResultComponent Visualizza metadati di classe documentale:
   - Codice e descrizione.
   - Periodo di conservazione.
   - Numero documenti in classe.
3. **Vantaggi dell'approccio**:
   - **Open/Closed**: aggiungere un nuovo tipo di risultato richiede solo di registrare il componente nella registry, senza modificare la factory.
   - **Single Responsibility**: ogni componente di visualizzazione conosce *solo* la struttura del proprio tipo di risultato.
   - **Performance**: solo il componente attivo è renderizzato; gli altri sono completamente distrutti (no lazy loading overhead).

6.3.4 Layer di ricerca globale

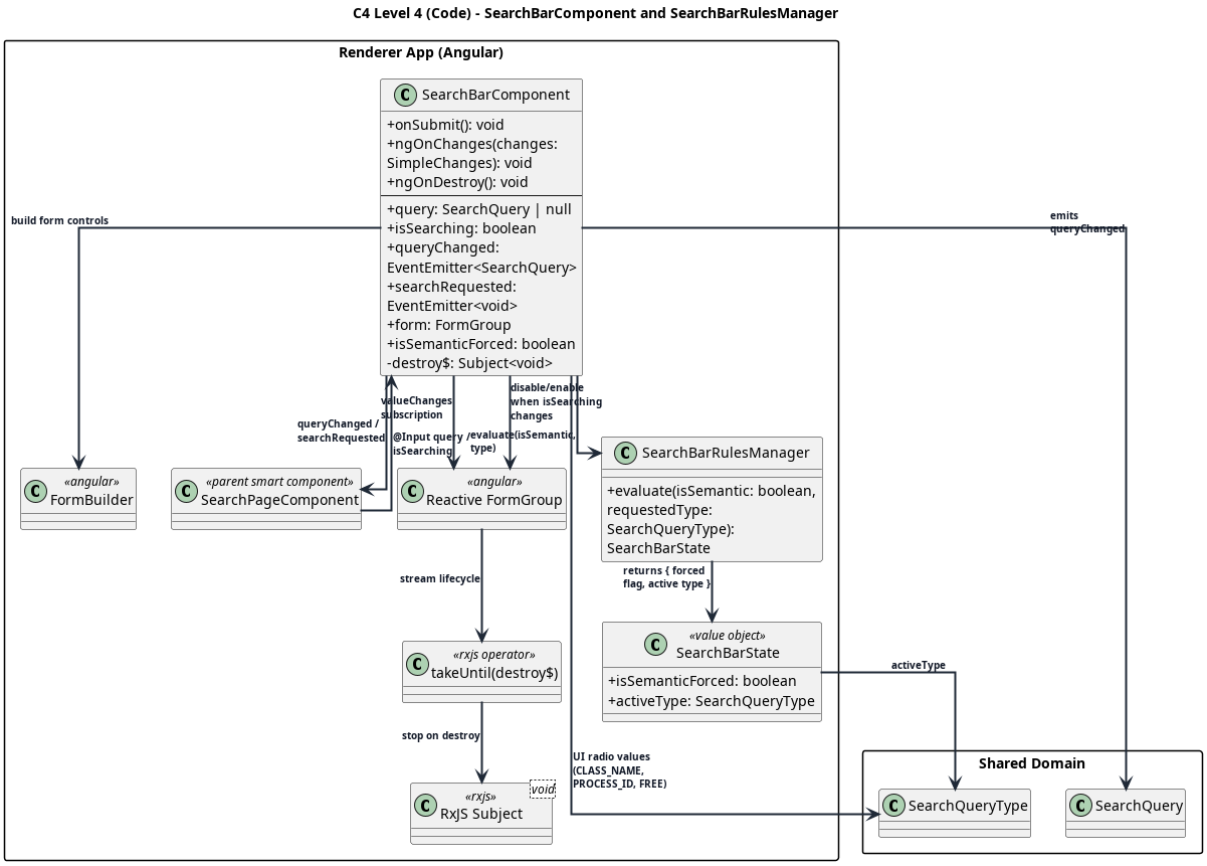

Figure 7: Diagramma C4 - SearchBarComponent e SearchBarRulesManager

SearchBarComponent e SearchBarRulesManager La search bar consente tre modalità di ricerca:

1. **Ricerca per ID**: inserimento dell'ID di processo; ricerca esatta nel back-end.
2. **Ricerca per nome**: inserimento di nome di classe o metadato; ricerca con wildcard.
3. **Ricerca semantica**: ricerca full-text su documenti usando embedding vettoriali (modello Xenova *paraphrase-multilingual-MiniLM-L12-v2*, eseguito nel browser tramite ONNX Runtime).

Il *SearchBarRulesManager* incapsula la logica di:

- Determinazione della modalità di ricerca in base all'input (euristiche: "se input match UUID pattern → ricerca ID").
- Abilitazione/disabilitazione della ricerca semantica per tipo di risultato. Attualmente è abilitata *solo* per documenti; estendere a processi richiederebbe configurazione nel servizio, non nel componente.

- Gestione dello stato di loading durante l'embedding vettoriale (che può impiegare millisecondi su CPU).
- Caching dei risultati (opzionale, per evitare query ridondanti).

Il componente di presentazione pura emette eventi:

- onSearch(searchTerm, mode): notifica la ricerca.
- onSemanticToggle(enabled): notifica l'abilitazione/disabilitazione della ricerca semantica.

Questi eventi sono catturati dal SearchPageComponent, che li instrada alla SearchFacade.

6.3.5 Conformità normativa e linee guida

Tutti i componenti di presentazione sono progettati per aderire all'Allegato 5 AGID. IN particolare per struttura e denominazione dei metadati, ordine dei campi, obbligatorietà La validazione della conformità è:

- **Strutturale**: il FilterRulesManager garantisce che solo i campi previsti da Allegato 5 siano disponibili.
- **Dinamica**: il FilterRulesManager può disabilitare campi in base al DocContext senza modificare il template.
- **Testabile**: ogni regola di filtro è una funzione pura, facilmente unit-testabile.

6.3.6 Comunicazione IPC e Gateway Layer

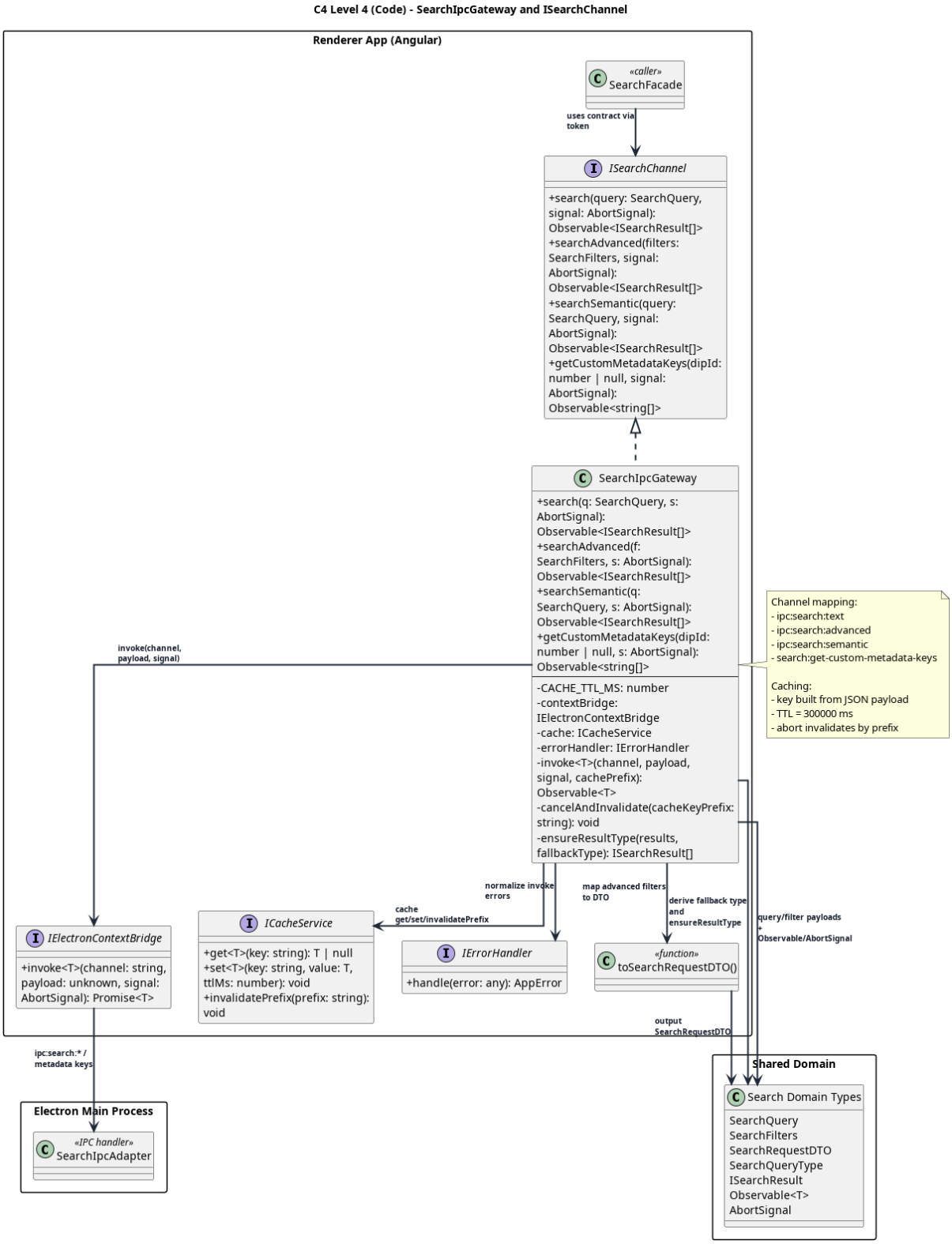

Figure 8: Diagramma C4 - SearchIpcGateway e comunicazione IPC

SearchIpcGateway e ISearchChannel Il SearchIpcGateway implementa ISearchChannel, fungendo da adapter tra il layer UI (SearchFacade) e il back-end tramite Electron IPC.

Responsabilità:

1. **Mapping DTO**: converte SearchQuery/SearchFilters (modello UI) in SearchRequestDTO (modello back-end) tramite toSearchRequestDTO().
2. **Caching**: mantiene cache a TTL 5 minuti per risultati di ricerca identici, riducendo overhead IPC.
3. **Error Handling**: cattura errori IPC e li trasforma in AppError normalizzati per il layer UI.

Interfaccia:

- search(query: SearchQuery, signal: AbortSignal): Observable<ISearchResult[]>
- searchAdvanced(filters: SearchFilters, signal: AbortSignal): Observable<ISearchResult[]>
- searchSemantic(query: SearchQuery, signal: AbortSignal): Observable<ISearchResult[]>
- getCustomMetadataKeys(dipId: number | null, signal: AbortSignal): Observable<string[]>

Flusso IPC

1. SearchFacade richiama gateway.search(query, abortSignal).
2. Gateway:
   (a) Verifica cache con cacheKey = hash(query).
   (b) Se HIT: restituisce cached results.
   (c) Se MISS: invoca contextBridge.invoke(SEARCH_CHANNEL, searchRequestDTO).
   (d) Aspetta risposta dal Main Process.
   (e) Salva in cache se successo.
   (f) Emette risultati via Observable.
3. SearchFacade aggiorna signal<SearchState>.results.
4. SearchResultsComponent visualizza dinamicamente tramite SearchResultFactory.

Integrazione con Core Backend Il backend riceve SearchRequestDTO e lo elabora tramite:

- SearchDocumentsUC: filtra documenti per metadati AGID.
- SearchProcessUC: filtra processi per ID/proprietà.
- SearchSemanticUC: embedding vettoriale (Xenova in browser o backend).
- SearchAggregateUC: filtra aggregazioni per proprietà DiP.

Ognuno restituisce ISearchResult[], che viene serializzato e rimandato al renderer via IPC.

Vantaggi dell'approccio

- **Separation of Concerns**: UI non conosce dettagli back-end, gateway non conosce dettagli UI.
- **Performance**: caching riduce IPC overhead.
- **Testabilità**: gateway è mockabile, backend è testabile in isolamento.
- **Resilienza**: error handling e abort signals gestiscono edge cases.

6.3.7 DTO e Trasformazione Dati

SearchRequestDTO vs SearchQuery La separazione tra SearchQuery (modello UI) e SearchRequestDTO (modello back-end) è fondamentale:

SearchQuery Rappresenta lo stato complessivo della ricerca in UI:

- query text, type, useSemanticSearch
- filters common, diDai, aggregate, subject, customMeta
- Contiene domini tipizzati (es., SubjectCriteria[], FilterField[]).
- Facilmente serializzabile per caching/logging.

SearchRequestDTO Rappresenta la richiesta per il back-end:

- Trasformazione via toSearchRequestDTO(searchQuery, filters).
- Ottimizzata per efficienza back-end: flattened, indici normalizzati.
- Non contiene dati UI-specifici (es., stati di UI, flag di caricamento).
- Utilizzata come chiave di cache nel gateway.

Mappers: SearchRequestMapper e CustomMetadataMapper

- *toSearchRequestDTO()*: converte SearchQuery SearchRequestDTO.
  - Flattens nested filters.
  - Normalizza subject criteria in formato back-end.
  - Aggiunge context headers (es., dipId, userId).
- *DocumentMetadataMapper*: converte raw metadata XML/JSON DocumentDTO.
  - Valida conformità AGID.
  - Estrae metadati standardizzati vs custom.

ISearchResult Union Type Il risultato di ricerca è una discriminated union:

- IDocumentSearchResult type: 'DOCUMENT', id, title, metadata, ...
- IProcessSearchResult type: 'PROCESS', id, docCount, ...
- IAggregateSearchResult type: 'AGGREGATE', id, docCount, ...
- IClassSearchResult type: 'CLASS', code, description, ...

Questa discriminazione permette al SearchResultFactory di istanziare il componente corretto, garantendo type safety.

6.4 Validazione Filtri

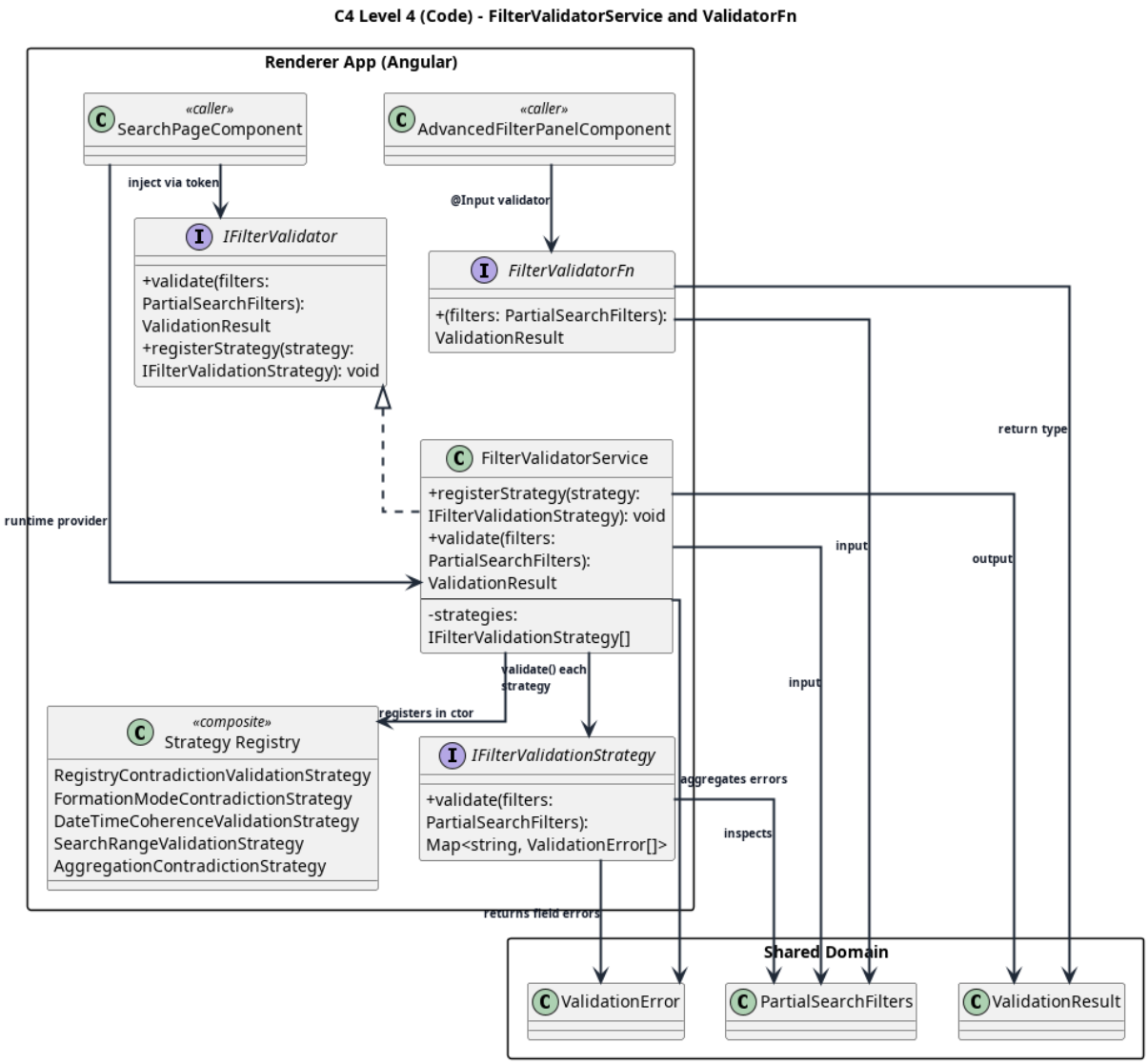

Figure 9: Diagramma C4 - FilterValidatorService e FilterValidatorFn

FilterValidatorService e FilterValidatorFn La validazione non è un aspetto casuale ma un layer dedicato. Il FilterValidatorService applica regole di validazione pure e composabili:

- **Pure Functions**: ogni regola è una funzione pura FilterValidatorFn = (filters) → ValidationResult.
- **Composizione**: regole si combinano (AND/OR) per validazioni complesse.
- **Error Mapping**: ValidationError[] contiene campo, messaggio, severity (ERROR | WARNING).

Esempi di regole:

- Se DiDaiFilters.tipologia è compilata, allora modalitàFormazione è obbligatoria.
- Se SubjectFilter include PERSONA_FISICA, allora codFiscale è obbligatorio.
- Se AggregateFilters.dataApertura è compilata, allora dataChiusura >= dataApertura.
- CommonFilters.annoInizio >= 1900 (vincolo AGID).

Inoltre anche in questo caso l'utilizzo di un pattern Strategy consente di applicare regole specifiche in base ai filtri presi in considerazione. Questo permette di aggiongere nuove regole senza modificare il codice esistente, ma semplicemente aggiungendo nuove funzioni di validazione e registrandole nella registry.

ValidationError e Stato di Errore Lo stato di ricerca mantiene una Map<fieldName, ValidationError> che:

- Permette visualizzazione inline degli errori nei componenti dumb.
- È aggiornata dinamicamente durante la compilazione (non solo al submit).
- È resettata prima di una nuova ricerca.